Université de Genève

Analyse et Traitement de l'Information
14X026

TP B: PCA, k-NN classification

Author: Engin Samet Dede

E-mail: Engin.Dede@etu.unige.ch

October 2025

UNIVERSITÉ DE GENÈVE
FACULTÉ DES SCIENCES
Département d'informatique

Exercise 1. Principal Component Analysis (PCA) Implementation

Please make a copy of this notebook and implement the missing methods. Upload the solved notebook.

Exercise 2. Principal Component Analysis

Randomly sample 5000 images of the digit 2 from the MNIST dataset. Denote this subset as $\mathcal{X} = \{x_i\}_{i=1}^n$, where each x_i is a 784-dimensional vector corresponding to a 28×28 image of the digit 2, and $n = 5000$. Perform PCA on $\mathcal{X}$. Denote the first m ($1 \leq m \leq 784$) principal components (with the largest variance) of $\mathcal{X}$ as $\{PC_1, \ldots, PC_m\}$. Note, that each PC_j is also a 784-dimensional vector.

If we imagine that we have a multidimensional data (for example, MNIST, each Picture has $28 \times 28 = 784$ dimensions), in this data PCA finds the dimensions those changing at most.

If we imagine a data in a space: Where data example is two-dimensional,

$$x = (x_1, x_2)$$

If we draw whole of this points in the graph, we will get a data cloud. Dimensions inside of this graph represents the extension of this data cloud. That extension is, if it extends towards a region and spreads towards that side. We call this direction of this graph. If we observe that the datas are spread into a one specific side we can call that the variance on this side is big. But if datas are collided onto a one specific point on a different side we can call that the variance on this side is smaller. The most stretched and spread side of the data represents the most changing direction of the graph and PCA is in favor to find this direction.

Mathematically:

$$v_1 = \arg\max_{\|v\|=1} Var(Xv)$$

Which means we are selecting a direction in a unit length v, and the projection variance in this direction should be maximum.

Here in this question inside of a MNIST image dataset (Modified National Institute of Standards and Technology dataset) we will select 5000 digit 2 image which is a data that has an equal possibility to select and they are random. By selecting using such method the selection representation of the data will represents the clusters character more accurately. Which we can say that random sampling is ensuring the subset to be more unbiased. Here in this question we have $(28 \times 28) = 784$ dimensional vector which represents the features of the data cloud.

Here in this question we are expected to find 5000 points which has 784 dimensions and this subset is called

$$X = \{x_i\}_{i=1}^n$$

X is a datasets that has been created from x_i's where $n = 5000$ and all $X = \{x_i\}_i \in \mathbb{R}^{784}$. Here the PCA will find the most changing directions in 5000 point with 784 dimension.

The output of the PCA will be the principal components (PC) = which shows the variance that has been explained above is bigger in which directions. In mathematics these are unit vectors which means that they show the direction. These dimensions are called principal components (PC). Each PC are like an indicator that explains the data in the best way axis.

Thus, mathematically PCA finds the covariance matrix's eigenvectors.

$$\Sigma v_j = \lambda_j v_j$$

Here the j represents the direction of the j'th principal component and λ_j shows the variance on this direction (size).

Here in this question with the quote "first m principal component" shows that if we use PCA on whole of the features we can get 784 different direction (PC). But we cannot claim that the all directions are not significant equally. That's why we are taking the first m of them which has the biggest variance.

$$\{PC_1, PC_2, \ldots, PC_m\}$$

PC_1 is the principal component that has the biggest variance. PC_2 is the second direction which is not repeating the first one so orthogonality represents the non-similarity of the principal components.

The first component represents the equation below:

$$v_1 = \underset{\|v\|=1}{\arg\max} Var(Xv)$$

But for the second one the equation should be represented as below:

$$v_2 = \underset{\|v\|=1,\, v \perp v_1}{\arg\max}\ Var(Xv)$$

and following whole of the principal components the equation below could be considered:

$$v_k = \underset{\|v\|=1,\, v \perp v_1, \ldots, v_{k-1}}{\arg\max} \quad Var(Xv)$$

Also, we are selecting the m in between $1 \leq m \leq 784$ because all of the images have 784 dimensions which represents 784 directions. But generally, some of the first ones, for example 10, 20, 50 represents the datas bigger side.

1. Centering vs standardization: Do you need to *center* the data (subtract the mean) and/or *standardize* (scale to unit variance) before PCA? Why or why not? Support your answer using formulas and a small sketch. Also discuss specifically for MNIST dataset.

2.1 (Answer)

Since PCA analyses the variance in the data it uses the covariance matrix.

$$\Sigma = \frac{1}{n}(X - \mu)^\top (X - \mu)$$

X: data matrix
μ: average of each row
Σ: examines how the variables are changing together according to themselves.

Reminder: Covariance matrix was utilized if we have more than one feature. PCA is using this because it should look all variable pairs covariance, since it is aiming to find the eigenvalues of the covariance matrixes.

For answering the question we should be aware of the definition of the centering and standardization.

Centering:

It tooks all of the features average and then subtracts this average from the data. Mathematically can be expressed as:

$$X_{centered} = X - \mathbf{1}\mu^\top$$

and by columns:

$$x_i^{centered} = x_i - \mu$$

Here with these equations, we can say that each column (feature) is centred to zero mean. Means that for each feature (pixel), we subtract its mean value across all samples, and every feature has a mean of zero. In example, if we have such points as (2,8), (3,9), (4,10) they are far from the origin and if we do the PCA without centring, Covariance direction will be calculated according to the origin. But the data is far from the origin and the direction will be wrong. That's why PCA should start from the data centre not from the origin.

If we also consider covariance definition:

$$Cov(x, y) = E[(x - E[x])(y - E[y])]$$

We are also considering the averages since the E represents the averages. Therefore, if we don't use centering the formula will malfunction and find PCA directions wrong.

Standardization:

it means that every feature (pixel), should be divided to its standard deviation

$$x_i^{standardized} = \frac{x_i - \mu}{\sigma}$$

We are using it because sometimes some features has so much different scales. Such as age and income. Age can have a range between 0–100 and also if we consider the income the range can be extended more such as 0–10000. Here the PCA will only consider the one with bigger variance, which is income, and neglect the age's variance. So PCA is sensitive to scale differences and in such situations the standardization should be done.

But when we consider MNIST each image has 784 feature and each pixel has the same scale (0–255 or 0–1) That's why there is no scale difference and all features are scaled in the same unit. But each pixels average can have difference have different average. For example, corner pixels are darker and they are close to zero but in the centre they are around 100).

If we consider that each pixel be represented as p_j where $j \in \{1, 2, \ldots, 784\}$, and let μ_j denote the mean intensity of pixel j over all n samples:

$$\mu_j = \frac{1}{n} \sum_{i=1}^{n} x_{ij}$$

Then, for border pixels p_b and center pixels p_c:

$$\mu_b \approx 0, \quad \mu_c \approx 100$$

Hence the variance of pixel values before centering is:

$$Var(x_{ij}) = E[(x_{ij})^2] - (E[x_{ij}])^2$$

If μ_j is large, the term $(E[x_{ij}])^2$ gains more importance for the variance computation, leading to biased PCA directions. After centering:

$$x_{ij}^{centered} = x_{ij} - \mu_j$$

And the new mean becomes:

$$E[x_{ij}^{centered}] = 0$$

Here variance becomes purely shape-dependent:

$$Var(x_{ij}^{centered}) = E[(x_{ij}^{centered})^2] = E[(x_{ij} - \mu_j)^2]$$

Thus, centering removes mean offsets and ensures that PCA captures genuine pixel intensity variations (shape structure) instead of brightness bias.

That's why we can claim that the Standardization is not necessary for the MNIST but the Centering can be required.

Here below for the better understanding the sketches are implemented:

Centering vs Standardization Before PCA

PCA aims to find the directions (principal components) along which the data varies the most. This means it analyzes the variance structure of the data through the covariance matrix:

$$\Sigma = \frac{1}{n}(X - \mu)^\top (X - \mu)$$

where X is the data matrix and μ is the mean vector of features.

—

Centering

Centering means subtracting the mean of each feature:

$$X_{centered} = X - \mathbf{1}\mu^\top$$

so that each feature has mean zero:

$$E[X_{centered}] = 0$$

Without centering, PCA directions are biased because the covariance matrix definition:

$$Cov(x, y) = E[(x - E[x])(y - E[y])]$$

explicitly assumes data are measured relative to their mean.

—

Standardization

Standardization rescales each feature to unit variance:

$$x_i^{standardized} = \frac{x_i - \mu}{\sigma}$$

It is required only when features are measured in different scales (e.g. *age* vs *income*). Otherwise, features with higher numerical ranges dominate the variance.

In the MNIST dataset, each pixel has the same unit and scale (0–255 or 0–1), so centering is needed, but standardization is not.

—

Mathematical reasoning for MNIST

Let each pixel intensity be x_{ij} (pixel j in image i). Then the mean per pixel is:

$$\mu_j = \frac{1}{n} \sum_{i=1}^{n} x_{ij}$$

Border pixels (background) have $\mu_j \approx 0$, while central pixels (where digits are drawn) have $\mu_j \approx 100$. Subtracting μ_j ensures each pixel contributes relative to its variation, not its brightness.

Since all pixels are in the same range (0–255), their σ_j are comparable, so dividing by σ_j (standardization) does not add information.

—

Graphical Illustration: Centering and Standardization in PCA

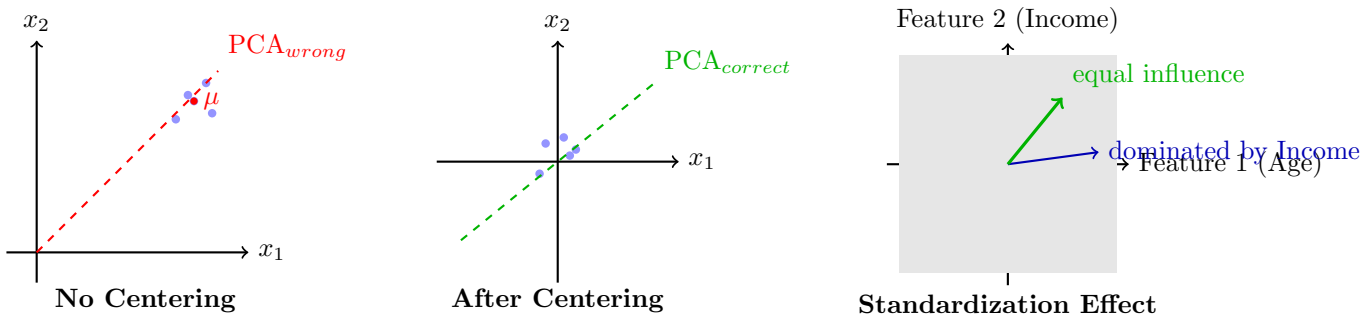


Figure 1: Left: Without centering, PCA direction is biased toward the origin. Middle: After centering, PCA finds the true direction of maximum variance. Right: Standardization ensures features with different scales (e.g., Age vs Income) contribute equally. For MNIST, only centering (middle case) is required.

2. Plot the sorted eigenvalues (descending) of the covariance matrix. Label axes clearly.

In this step, we perform PCA on the MNIST dataset, focusing on the digit "2". We compute the covariance matrix, perform eigenvalue decomposition, and plot the sorted eigenvalues. Each eigenvalue λ_j represents the variance captured by the jth principal component.

$$\Sigma = \frac{1}{n}(X-\mu)^\top(X-\mu)$$
$$\Sigma v_j = \lambda_j v_j$$

The following Python code illustrates these steps:

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml

#Here we are going to perform PCA on the MNIST dataset, specifically focusing on
#the digit '2'. We will compute the covariance matrix,
# perform eigenvalue decomposition, and plot the sorted eigenvalues.
X_all, y_all = fetch_openml('mnist_784', version=1, return_X_y=True, as_frame=False)
X2 = X_all[y_all == '2']          # (N2, 784)

# We are selecting 5000 random samples of the digit '2' for PCA analysis.
rng = np.random.default_rng(42)
idx = rng.choice(len(X2), size=5000, replace=False)
X = X2[idx].astype(np.float64)    # (5000, 784)

# This steps does centering the data which means subtracting the
# mean, computing the covariance matrix,
mu = X.mean(axis=0, keepdims=True)
Xc = X - mu

# Covariance (1/n or 1/(n-1) it doesn't matter; most statistics use n-1)
n = Xc.shape[0]
Sigma = (Xc.T @ Xc) / n  # or / (n-1)

#Eigen decomposition of the covariance matrix
#Eigenvalues are computed using the eigh function, which is more stable for symmetric matrices like covarian
# Sigma is symmetric, so we use eigh
eigvals, eigvecs = np.linalg.eigh(Sigma)    # comes in ascending order
eigvals = eigvals[::-1]                     # here comes in descending order
# eigvecs = eigvecs[:, ::-1]                # if needed later the directions can be reversed too

# Drawing
plt.figure(figsize=(7,4))
```

```
plt.plot(np.arange(1, len(eigvals)+1), eigvals, marker='o', linewidth=1)
plt.xlabel("Component index - j")
plt.ylabel("Eigenvalue $\\lambda_j$ (variance)")
plt.title("Sorted eigenvalues of the covariance matrix for digit '2'")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Initial eigenvalues are high because most of the variance is captured in a few principal components at the beginning. The contribution of the subsequent elements is observed to be less, and this is clearly evident from the flattening of the curve of the graph. Here at the below graph we can observe the same situation: To solve this question, we must first understand the following steps in detail: - PCA first measures the

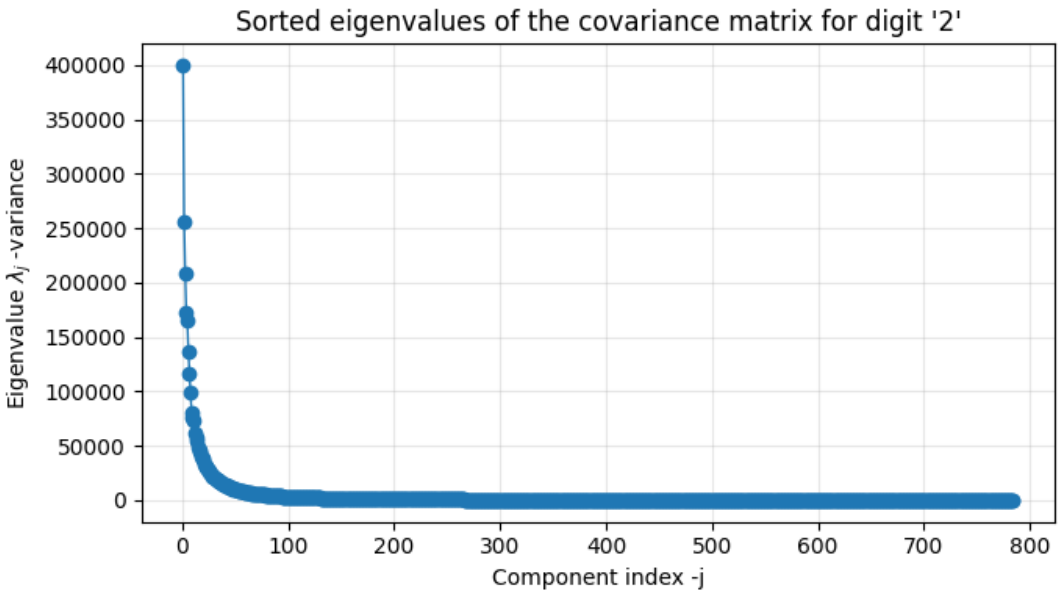


Figure 2: eigenvalues of the covariance matrix created for digit '2'.

variance of the data, - Then it finds the directions where this variance is greatest, - It expresses the importance of these directions with eigenvalues. In other words, the eigenvalues indicated on the graph are critical for understanding the directions. To explain this question mathematically, the following steps can be observed: We take care to select the data so that the rows are examples and the columns are features. To do this we are using transpose.

$$X = \begin{bmatrix} x_1^\top \\ x_2^\top \\ \vdots \\ x_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}, \qquad d = 784,\ n = 5000,$$

What we see here is the flattened vector of each $x_i \in \mathbb{R}^d$ image. The flattened vector aims to keep the matrix of digits, which is normally a 28×28 image in MNIST, in a single row.

$$x_i = \begin{bmatrix} p_{1,1} & p_{1,2} & \cdots & p_{1,28} \\ p_{2,1} & p_{2,2} & \cdots & p_{2,28} \\ \vdots & \vdots & \ddots & \vdots \\ p_{28,1} & p_{28,2} & \cdots & p_{28,28} \end{bmatrix}$$

with flattened vector the matrix above will be seen as below:

$$x_i = \left[p_{1,1}, p_{1,2}, \ldots, p_{1,28}, p_{2,1}, p_{2,2}, \ldots, p_{28,28}\right]^\top$$

from now on $x_i \in \mathbb{R}^{784}$. We call this flattening because we are making 2D matrix a 1D vector.

In the second stage, we need to center because PCA measures the variance around the mean and for this reason it is necessary to first extract the mean of each feature.

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i \in \mathbb{R}^d.$$

$$\widetilde{X} = X - \mathbf{1}\,\mu^\top \in \mathbb{R}^{n\times d},$$

Here $\mathbf{1} \in \mathbb{R}^n$ is a vector whose components are all 1. And Therefore,

$$\frac{1}{n}\sum_{i=1}^{n} \tilde{x}_i = 0.$$

The covariance matrix is used to measure how the properties of the data change together. Here below I computed the covariance matrix. In the case of centered data, the sample covariance takes the following form:

$$\Sigma = \frac{1}{n}\,\widetilde{X}^\top \widetilde{X} \in \mathbb{R}^{d\times d}.$$

Others use 1 divided by $n-1$; this does not alter the directions (eigenvectors) returned by PCA, but does slightly change the scale. The diagonal terms of the Sigma matrix are the variances of each of the features whereas the non diagonal terms are also the joint variation (covariance) of the two features. Moreover, the symmetricity and semi-positive definition of the elements of the set of matrices should be satisfied as like below:

$$v^\top \Sigma v \;=\; \frac{1}{n}\,\|\widetilde{X}v\|^2 \;\geq 0.$$

This also makes sure that the eigenvalues are not negative.

On the other hand, the principle of PCA is to determine the directions of the maximum change of the data (variance). Moreover, these directions form the eigenvectors of the matrix of the Sigma and the variance in the directions is the eigenvalues. Thus, I obtained the eigenvalue problem:

$$\Sigma\, v_j \;=\; \lambda_j\, v_j, \qquad j = 1, \ldots, d.$$

In this instance, v_j signifies the direction of the j th principal component whereas λ_j signifies the amount of the explained variance in that direction. The eigenvectors are orthogonal (they are selected to be orthogonal), and the eigenvalues. I ordered all the eigenvalues that this solution gave me according to their size:

$$\lambda_1 \;\geq\; \lambda_2 \;\geq\; \lambda_3 \;\geq\; \cdots \;\geq\; \lambda_d \;\geq\; 0.$$

The rationale of this ranking is to easily observe which elements define the highest share of the total variance in the data. The first principal eigenvalue is the largest eigenvalue of the eigenvalues which is denoted as 1, the second principal eigenvalue is the second largest eigenvalue which is denoted as 2 and so on.

To graph the eigenvalues on the graph, I plotted the component index ($j = 1, 2, \ldots, d$) on the horizontal axis, the magnitude of the eigenvalue eigenvalue (λ_j) on the vertical axis and joined up the points with a line. This plot is referred to as a scree plot. This graph is usually characterized by the large values of the first few eigenvalues and then the values drop at a high pace to the end, becoming level. This means that a significant amount of difference between the data is captured in a few major components. To substantiate this observation further, I can also list the cumulative explained ratio of the variance:

$$\mathrm{CEV}(m) = \frac{\sum_{j=1}^{m} \lambda_j}{\sum_{j=1}^{d} \lambda_j},$$

This ratio tends to 1 with increasing m answering the question, "How much of the total variance is explained by the first m components?" As a validation, I would know that the trace of the covariance matrix is equal to the sum of all the eigenvalues:

$$\mathrm{tr}(\Sigma) = \sum_{j=1}^{d} \Sigma_{jj} = \sum_{j=1}^{d} \lambda_j.$$

The equation is applicable in testing the consistency of calculations.

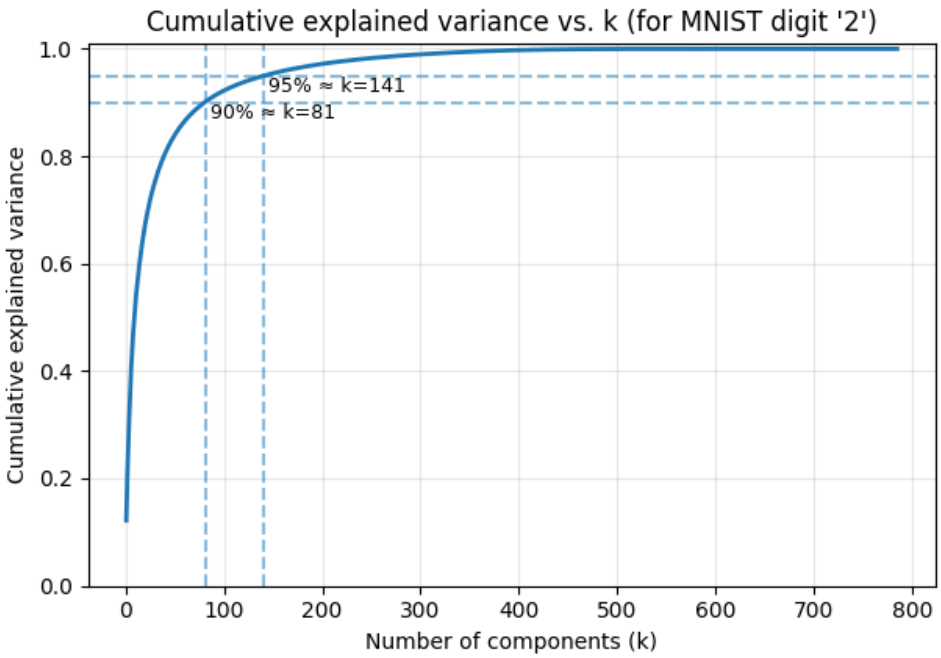
I briefly mentioned the reason I had used the centering first, namely MNIST. As pixels are all measured in the same scale and with an equal measurement unit (0–255 or 0–1) there will be no change in scale; hence, no standardization is required to change the direction of PCA. The average brightness of pixels may however vary as location (background pixels are averagely lower, near the center pixels are averagely higher). Pca done without the removal of this offset can take the brightness offset as variance. That is why centering is necessary; the eigenvalues are indeed a measure of the shape/variance information.

3. Plot the cumulative explained variance as a function of the number of components k: $\sum_{j=1}^{k} \lambda_j / \sum_{j=1}^{784} \lambda_j$, where λ_j are eigenvalues sorted in descending order.

When I run the code lab2.3.py I got the outputs and graph below: Output:

$$\text{Top 10 variance ratios (\%): } [12.14,\ 7.84,\ 6.34,\ 5.23,\ 4.96,\ 4.15,\ 3.45,\ 3.01,\ 2.45,\ 2.32]$$
$$k_{\text{for 90\% variance}} = 81$$
$$k_{\text{for 95\% variance}} = 141$$

also when the results are plotted the graph below was obtained: To explain the sense of working of the



code we should understand the procedure below:

```
X = get_twos(5000, seed=42)
```

Here above we are creating the raw input of the PCA. All 2 digits in MNIST has been created using 784 features so we can imagine the data matrix as:

$$X = \begin{bmatrix} — & x_1^\top & — \\ — & x_2^\top & — \\ & \vdots & \\ — & x_n^\top & — \end{bmatrix} \in \mathbb{R}^{n \times d}$$

Where n is 5000 and d is feature (784). Each row is an image and each coloumn is the gray colour value.

```
astype(float)/255.0
```

with this row of the code the pixels are normalized to the 0-1 interval.

Secondly, the code was centered using the lines below:

```
mu = np.mean(data, axis=0, keepdims=True)
data_c = data - mu
```

PCA would like to find the variance. But data's general average (in example the whole pixels average brightness) does not considered as an important thing. Therefore, we are centering the data over average.

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad X_c = X - \mathbf{1}\mu^\top$$

Here can be explained like below:

- μ: every column average (size d)
- $\mathbf{1}$: $n \times 1$ vector of 1s in size
- X_c: centered data (each column average ≈ 0)

Here below the code calculates the covariance:

```
cov = (data_c.T @ data_c) / (n - 1)
```

Covariance measures how two variables change together. And after that PCA gathers this information for all pairs of variables. The formula that has been used in the code is written below:

$$C = \frac{1}{n-1} X_c^\top X_c$$

Here, $C \in \mathbb{R}^{d\times d}$. That is, for each C_{ij}, it gives the joint variation (covariance) of column i and column j. Simply put:

C_{ii} is the variance of the feature itself (how much it changes), $\quad C_{ij}$ is how much two features change together.

C is symmetric ($C = C^\top$), meaning that it is equal to each other and positive semi-definite with its transpose (all eigenvalues ≥ 0). Therefore, using `np.linalg.eigh` in NumPy is more stable. I used it because it is specifically optimized for symmetric matrices.

After that I have found the eigenvalues and eigenvectors using the code line below:

```
vals, vecs = np.linalg.eigh(cov)
```

I wanted to solve the covariance matrix as follows and implemented it in the code:

$$Cv_j = \lambda_j v_j$$

The operation performed here is the eigenvalue–eigenvector decomposition in linear algebra.

v_j : a "direction" (principal component) of the data, $\quad \lambda_j$: the amount of variance explained in that direction.

In other words, the larger the λ_j, the more information the v_j direction carries. Therefore, we sort the values in descending order with `np.argsort(vals)[::-1]`:

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d$$

Then, we sort the eigenvectors in the same order with `vecs[:, order]`. It's important to mention a fundamental property of PCA in mathematics at this point: The v_1 direction is the direction of maximum variance in the data. The v_2 direction is the direction perpendicular to v_1 and captures the second largest variance, and so on. These considerations has been observed initially at the beginning of the question

The following line of code was written to calculate the cumulative explained variance in the next stage of the problem.

```
cev = np.cumsum(eig_val) / eig_val.sum()
```

CEV is the ratio we use when we want to measure how much total variance we preserve based on the number of components we are looking for.

I wanted to solve the covariance matrix as follows and implemented it in the code:

$$Cv_j = \lambda_j v_j$$

The operation performed here is the eigenvalue–eigenvector decomposition in linear algebra.

v_j : a "direction" (principal component) of the data, λ_j : the amount of variance explained in that direction.

In other words, the larger the λ_j, the more information the v_j direction carries. Therefore, we sort the values in descending order with `np.argsort(vals)[::-1]`:

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d$$

Then, we sort the eigenvectors in the same order with `vecs[:, order]`. The sense of PCA in mathematics at this point could be considered to understand more accurately. The v_1 direction is the direction of maximum variance in the data. The v_2 direction is the direction perpendicular to v_1 and captures the second largest variance, and this procedure continues.

Then the graph was created to show the PCA and MNIST datasets 2 digit variance distribution. The horizontal represents the number of components k, whereas the vertical axis is followed by the ratio of cumulative explained variance (CEV). The curve initially steeply increases, and then begins to start flattening out, and this only indicates to us that the first few elements contain most of the information of the data.

As an example, only 81 items describe 90 percent of the total variance and 141 will take us to 95 percent. Thus it may be that though the original space was 784-dimensional we can reduce it to about 80–140 dimensions and still retain the essentials. Moreover, the row of variance ratios with the top 10 in the output verifies that the top 10 components significantly explain the variance of approximately 52 per cent. To sum up, PCA reduces high dimensionality retaining the main patterns, which provides a strong step forward in compression data as well as computing speed.

4. Any image $x_i \in \mathcal{X}$ can be reconstructed using $\{PC_1, \ldots, PC_m\}$ by

$$r_m(x_i) = \sum_{j=1}^{m} \langle x_i - \bar{x}, PC_j \rangle \cdot PC_j + \bar{x}$$

where $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$ is the average image and $\langle \cdot, \cdot \rangle$ denotes the inner product. The corresponding reconstruction error of $\mathcal{X}$ (using the Euclidean norm) is

$$err(\mathcal{X}, m) = \frac{1}{n}\sum_{i=1}^{n} \|x_i - r_m(x_i)\|_2.$$

- Plot $err(\mathcal{X}, m)$ as a function of m for $m = 0, 1, \ldots, 784$ (include $m = 0$ as the baseline, which yields $r_0(x_i) = \bar{x}$).
- Define the relative error $\mathrm{RE}(m) = err(\mathcal{X}, m)/\max_k err(\mathcal{X}, k)$. Find the smallest m such that $\mathrm{RE}(m) \leq 0.50$, $\mathrm{RE}(m) \leq 0.05$, and $\mathrm{RE}(m) \leq 0.01$ (i.e., 50%, 95%, and 99% reduction relative to the maximum error).

When performing PCA, we first subtract the sum of the arithmetic mean of the vectors from each image, thus achieving centering. When the eigenvalue and eigenvector decomposition of the covariance matrix is performed on the centered data, we obtain 784 unit-length, perpendicular lines. I've learned that these are called the principal components, but I'm writing them down again to remind myself. As far as I've learned, these vectors are orthonormal, and this orthonormality means that the vector satisfies both the orthogonality

$$\langle PC_j, PC_k \rangle = 0$$

and the unit norm

$$\|PC_j\| = 1$$

conditions. Also for all PC_j we can get eigenvalue λ_j, λ_j represents the variance in that direction, which can be also explained as the "energy" of the data in that direction, and is generally arranged from largest to smallest, in the order

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_{784}.$$

To check whether the PCA summarizes the data properly, reconstruction shows how well the summary reproduces the original data. Reconstruction error numerically measures the success of this summary.

We can express the reconstruction as like the below equation using m Principal Components:

$$r_m(x_i) = \bar{x} + \sum_{j=1}^{m} \langle x_i - \bar{x}, PC_j \rangle PC_j.$$

This formula has been utilized inside of the reconstruction error formula, that is, we quantify the squared distance between an image and its m-component approximation. To calculate MSE:

$$err(X, m) = \frac{1}{n} \sum_{i=1}^{n} \|x_i - r_m(x_i)\|^2$$

Since the $\{PC_1, PC_2, \ldots, PC_{784}\}$ is orthonormal we can write $x_i - \bar{x}$ vector with all Principal Components.

$$x_i - \bar{x} = \sum_{j=1}^{784} \langle x_i - \bar{x}, PC_j \rangle PC_j$$

If we take only first m ones, the remaining ones become residual component:

$$x_i - r_m(x_i) = \sum_{j=m+1}^{784} \langle x_i - \bar{x}, PC_j \rangle PC_j$$

This is now over $PC_{m+1}, \ldots, PC_{784}$ direction and since these directions are orthogonal, squared norms can be summated like below:

$$\|x_i - r_m(x_i)\|^2 = \sum_{j=m+1}^{784} \langle x_i - \bar{x}, PC_j \rangle^2.$$

Now if we take average over whole of the samples, mean square error is equal to:

$$err(X, m) = \sum_{j=m+1}^{784} \lambda_j.$$

We draw another critical conclusion from this formula. The largest error occurs when $m = 0$, because then we use no principal components and only the average image. Indeed, for $m = 0$, the error is

$$\sum_{j=1}^{784} \lambda_j,$$

which is the total variance; this is our reference for the "maximum error."

If relative error was written like below:

$$RE(m) = \frac{\max_k err(X, k)}{err(X, m)} = \frac{\sum_{j=1}^{784} \lambda_j}{\sum_{j=m+1}^{784} \lambda_j}$$

We can understand that this is

$$1 - CEV(m)$$

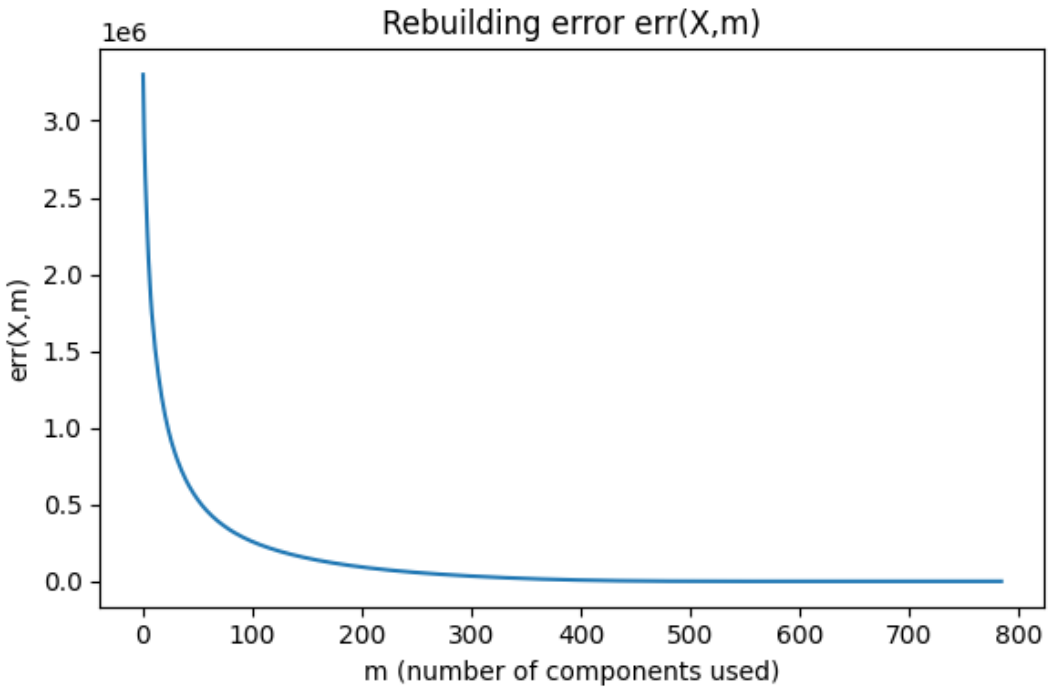
As a result,

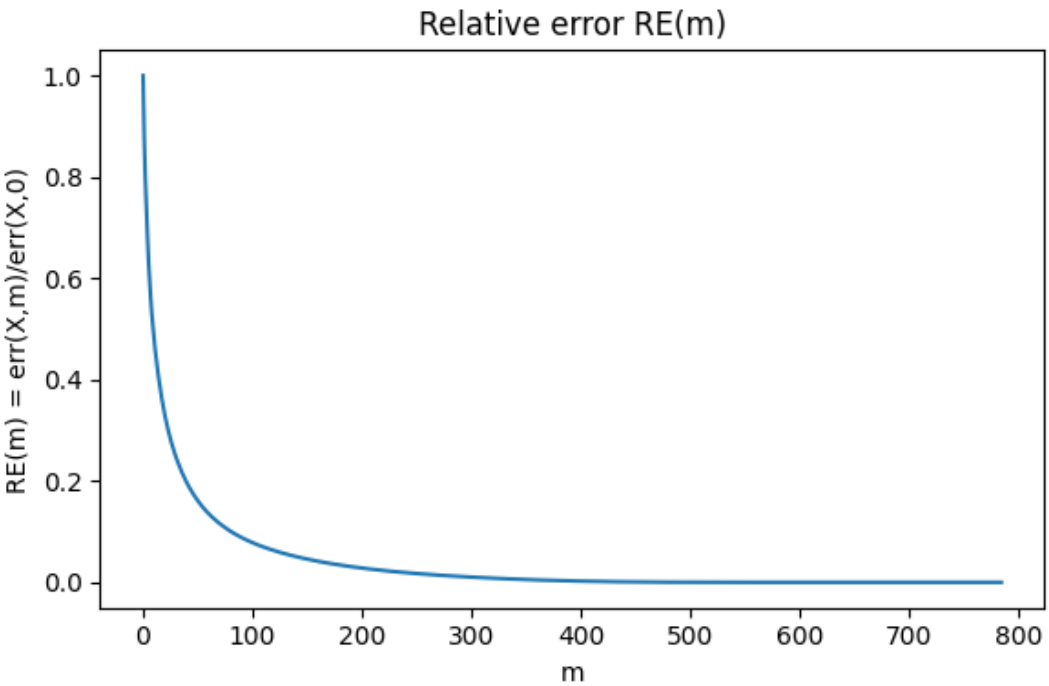
$$RE(m)0.50 \Rightarrow CEV(m)0.50,$$
$$RE(m)0.05 \Rightarrow CEV(m)0.95,$$
$$RE(m)0.01 \Rightarrow CEV(m)0.99.$$

Thus, in order to obtain the lowest possible m values to achieve the desired thresholds, all we have to do is left-right scan the $CEV(m)$ curve and take the point where the desired thresholds are first crossed.

For the graphs, the initial one shows the Rebuilding error $err(X, m)$. In the beginning $m = 0$ there is no component utilized and the error is the highest. When m is used gradually, the graphic's error value shrinks. And when $m \geq 300$ error is near zero. Which means instead of 784 dimensions, with only ~ 300 features we can represent $\sim 99\%$ of the images.

The second one shows the Relative Error $RE(m)$, which shows how much the error is reduced compared to the initial situation. At the beginning it is equal to 1 and it gets lower and lower when m increases. And the output of the code represents the decrease of the $RE(m)$ with more m components.





And the outputs below shows when the m components increasing the RE is getting smaller.

$$RE(m) \leq 0.50 \quad \Rightarrow \quad m = 10,$$
$$RE(m) \leq 0.05 \quad \Rightarrow \quad m = 142,$$
$$RE(m) \leq 0.01 \quad \Rightarrow \quad m = 303.$$

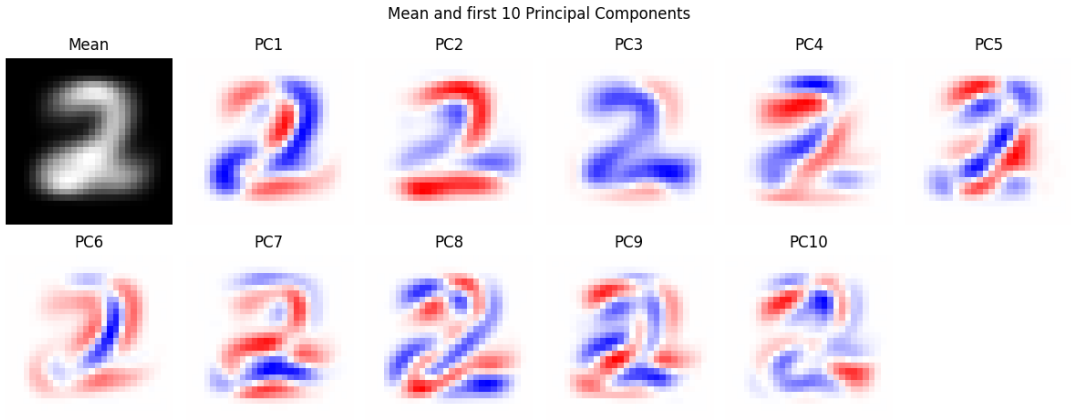
5. Compare the three plots (eigenvalues, cumulative explained variance, and reconstruction error). What relationships do you observe?

Thet are the three graphs that completely complement each other: The eigenvalue graph shows the variance that each component has; the sum of the eigenvalues leads to the cumulative explanatory variance curve; and the sum of this amount (1 -CEV) is the reconstruction error graph.

As a summation, the larger the eigenvalues the faster the eigenvalues decrease and the higher CEV increases, and the smaller the error in reconstruction.

6. Plot the mean vector of $\mathcal{X}$ and the first 10 principal components as 28×28 images (and not 1D vectors).

Here in the below representation of the 10 principal components the lab2.5.py file has been utilized. the digits are also coloured using red, blue and white to represent the meanings below: Red: pixel value is increasing Blue: pixel value is decreasing White (0) = No change around the average pixel value.

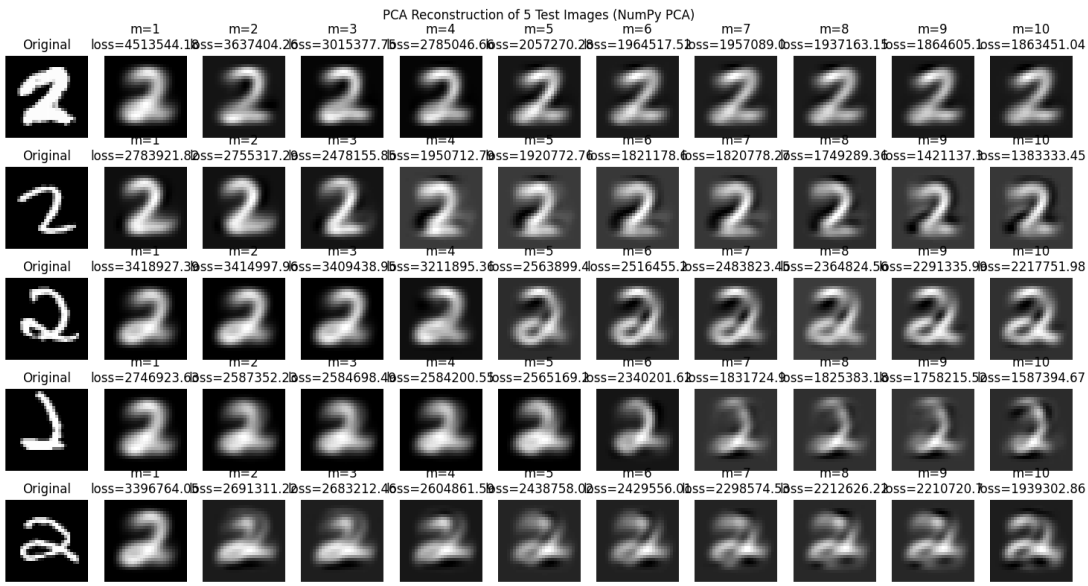


7. Sample 5 random images of digit 2 from MNIST (outside of $\mathcal{X}$; use a held-out set). For each image $\mathcal{I}$ and for each $m = 1, 2, \ldots, 10$ do:

   - Compute the reconstruction of $\mathcal{I}$ using the first m principal components of $\mathcal{X}$
   - Calculate the reconstruction loss $\|\mathcal{I} - r_m(\mathcal{I})\|_2$ for each image and each m (round the loss values to two decimal places).

   In total, you should have 50 reconstructed images and 5 original images. Plot these images in a grid/table (e.g., columns: *original image, reconstructions for* $m = 1, 2, \ldots, 10$) Don't forget to add reconstruction loss.

When the code lab2.7.py has been utilized the below result will be achieved:



Exercise 3. k-Nearest-Neighbour (k-NN) Classification

Randomly sample a **balanced** training set of 5000 images from the digits "0", "1", "2", "3", and "4" (1000 per class). For the testing set, use all available testing images of the same classes. Record the random seed you used.

The main idea of *1*-NN classifier is:

To classify a sample x in the testing set, select the nearest sample from $\mathcal{X}$ in the *training set*, then predict the class label of x to be the same as this nearest sample.

To generalize this idea to k-NN classifier you need to select k nearest samples instead of only one.

1. Do you need to *center* the data (subtract the mean) and/or *standardize* (scale to unit variance) before k-NN? Why or why not? Support your answer using formulas and a sketch. Also discuss specifically for MNIST dataset.

   k-NN is an algorithm that has been utilized for the machine learning and it is basic but it has a strong sample base and that means instead of teaching the model it decides directly with data examples.

   For example, if we would like to guess a class of a sample:

   First we will calculate the distance between the whole samples in the training subset.

   Then we will select the k amount of neighbours (samples).

   Finally, from this k neighbour we will assign the majority class to the test sample as a guess.

   As a result k-NN is an algorithm that relates the new vector with the most similar class of vectors.

   Mathematically,

$$d(x, x_i) = \sum_{j=1}^{p} (x_j - x_{ij})^2$$

   for each training sample we should calculate the distance.

   Then we select the smallest k amount of them:

$$N_k(x) = \text{closest } k \text{ amount of neighbours}$$

   Then we should look each class to find the most similar one testing each vector.

$$\hat{y}(x) = \text{mode}\{y_i : x_i \in N_k(x)\}$$

   For the k-NN equation each feature's scale directly affects the distance, bigger values are intermittent or variance of the bigger features gives more weight to the distance.

   Centering: If we center the slope and test data with same mean vector μ:

$$\tilde{x} = x - \mu, \quad \tilde{y} = y - \mu$$

$$\|\tilde{x} - \tilde{y}\|_2 = \|(x - \mu) - (y - \mu)\|_2 = \|x - y\|_2$$

   That means the Euclidean distance is translation invariant since when the data space is slide one vector away it will not change.

$$d(x, y) = d(x + c, y + c)$$

   So centering will not change the difference between whole couples and k-NN order will not change.

   But there is an exception, and that is that if a different centering is applied to only the training or only the test side (μ's are different), the distances will be distorted.

   Standardization:

   For feature j:

$$z_j = s_j(x_j - \mu_j), \quad s_j = \text{Var}(X_j)$$

   and Euclidean distance, after standardization:

$$\|z - w\|_2 = \sum_{j=1}^{p} (s_j x_j - y_j)^2$$

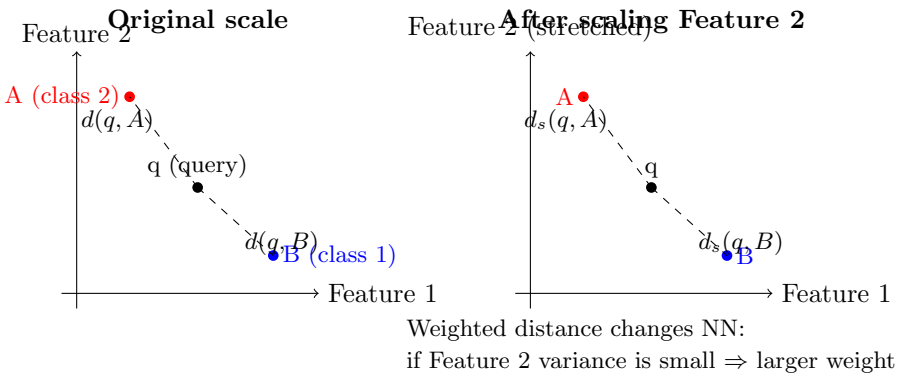
What we are doing then is weighing each feature by $\frac{1}{\text{its variance}}$. In other words, $\frac{1}{1}$. Also we can say that, this causes the high-variance features to be dragged down and moved up the low-variance features. Consequently, the sequence of adjacent points may change and the boundary dividing classes may move.

When the scales are very different, e.g. feature 1 has values between 0 and 10,000 and feature 2 has values between 0 and 1, then we actually require standardization or the big-scale feature will take up all the distance measure.

In cases where all things are measured in the same units and the variances are more or less similar, standardizing does not actually proceed much.

Considering that a mere division of every pixel by 255 scales distances by the same constant, within the same way the ranking remains unchanged. Simply, the transformation of a [0,255] image into [0,1] image does not influence the neighbours of k-NN.

Here in the below graphs we can observe the situation illustrated:



With this illustration we can observe that how the feature got bigger or smaller when it is standardized and its effect on the k-NN. In the beginning the feature 1 and feature 2 had the same effect but after standardization feature 2 after standardization became bigger so direction is more effective for it. In this situation A could be considered closer.

For MNIST: global centering does not affect the k-NN result. So Centering is irrelevant for the MNIST.

But for standardization, z-score which is a standardization considered:

$$x' = \frac{x - \mu}{\sigma}$$

Bigger variance ⇒ smaller weight

Smaller variance ⇒ bigger weight

This might be a bad idea because in MNIST most of the background is black and the background pixels variance is generally close to 0.

If we apply z-score to them $\sigma \approx 0$, then $\frac{1}{\sigma^2}$ becomes very large, and the result of the equation above will become so big that causes the background noise to become dominant.

The best approach for MNIST is normalizing. We should bring pixel values from 0–255 to 0–1:

$$x_{\text{norm}} = \frac{x}{255}$$

This will help all values to be in the same scale. We will not need to do any centering or standardization after using normalization.

2. Compute the baseline *1*-NN classification accuracy, defined as the percentage of samples correctly classified in the testing set.

1-NN Classification Explanation

I imagined that we have a training set labelled such as

$$X = \{(x_i, y_i)\}.$$

Here in this equation, x_i is the feature vector for each image. And for y_i we can say that it is the class labels.

1-NN classing is for the x sample in the test, the whole samples will be considered with their distance to the x. With the label of the vector that has smallest distance the y will be guessed.

Distance measurement: we will use Euclidian distance which is the most common one. We should convert images to vectors then calculate the distance between two vectors:

$$\|x - x_i\|_2.$$

Using the SEED = 20251020 inside of the MNIST 0-4 subsets, we can get 1000 samples from each class and we take those samples balanced. Using balanced 5000 training set and the all test samples and 1-NN (Euclidian) run together, we will get the test accuracy.

These equations has been compiled inside of thelab3.2.py. It can be observed for more information. After the real and predicted labels used in the accuracy_score function below:

$$\text{acc} = \text{accuracy_score}(y_test, y_pred) \times 100$$

the result is achieved as 98.09

Class-wise accuracy report:

	precision	recall	f1-score	support
0	0.9827	0.9942	0.9884	1714
1	0.9704	0.9944	0.9823	1977
2	0.9751	0.9546	0.9647	1761
3	0.9882	0.9734	0.9808	1806
4	0.9905	0.9874	0.9890	1587
accuracy			0.9809	8845
macro avg	0.9814	0.9808	0.9810	8845
weighted avg	0.9810	0.9809	0.9808	8845

True positive: Represents the model correctly predicting "2" for something that is actually "2". False Positive (FP): It actually belongs to another class, but the model incorrectly predicted it as "2". False Negative (FN): The true value is "2," but the prediction is something else.

Precision It examines how appropriate it is for the model to choose this class.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall It examines how much of what is true 2 is considered correct. It checks to see if any are missed.

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 score A balanced average of Precision and Recall. It tests whether the precision or recall doing their job accurately.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

support is the number of test samples belonging to class.

Macro Average It is the simplified average of the precision, recall, and F1 scores of each class. That can be reffered as, all classes are considered equally weighted.

$$\text{Macro Avg} = \frac{1}{N}\sum_{i=1}^{N}\text{Metric}_i$$

Weighted Average Unlike the macro average, the weighted average is calculated by weighting according to the number of supports. It aims to provide more accurate results in case the data is unbalanced.

$$\text{Weighted Avg} = \frac{\sum_i(\text{Metric}_i \times \text{Support}_i)}{\text{Total Support}}$$

When all these data are examined in the table, a consistent result can be observed. Also the confusion matrix shows how many values are guessed correctly.

Confusion matrix:

$$\begin{bmatrix} 1704 & 1 & 7 & 0 & 2 \\ 0 & 1966 & 6 & 2 & 3 \\ 23 & 33 & 1681 & 19 & 5 \\ 4 & 11 & 28 & 1758 & 5 \\ 3 & 15 & 2 & 0 & 1567 \end{bmatrix}$$

Here in the first line we are checking wheter the 0's are guessed correctly 1704 of them are correct and 1,7,2 are guessed as 1,2 and 4.

3. (PCA + *1*-NN) Repeat the following for each

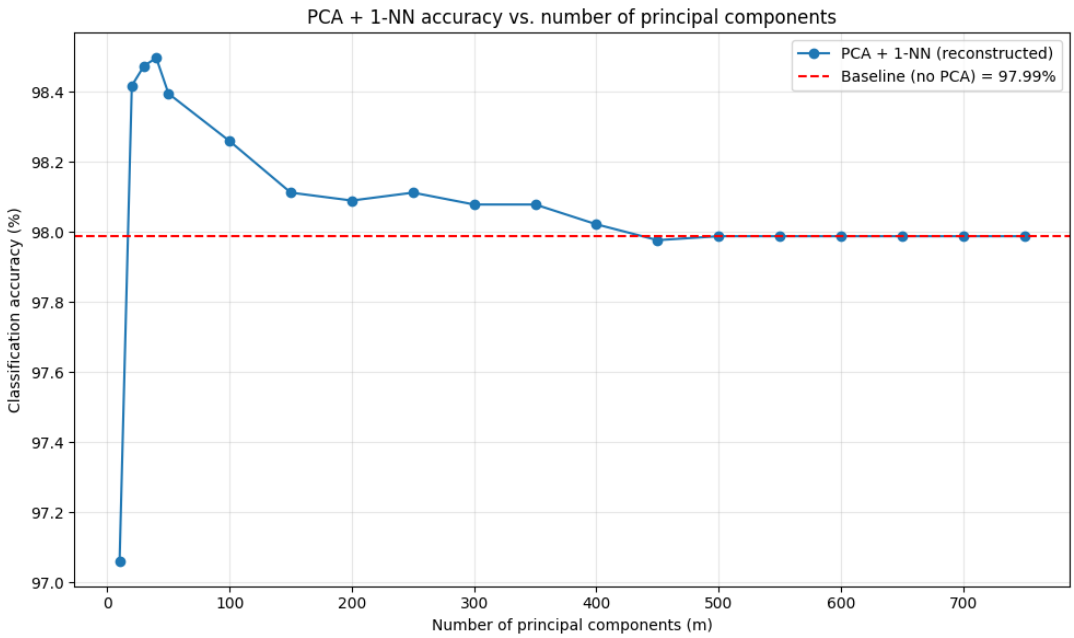
$$m = 10, 20, 30, 40, 50, 100, 150, 200, 250, ldots, 750$$

- Fit PCA *only on the training set*, then reconstruct the training images using the first m principal components $\{PC_1, \ldots, PC_m\}$ (i.e., use `transform` to m dimensions and then `inverse_transform` back to image space).
- For each testing sample x, compute its reconstruction $r_m(x)$ using the same PCA (transform to m and inverse-transform back to image space).
- Classify x by finding its nearest *reconstructed* training sample (Euclidean distance in the image space) and using that label (*1*-NN).

Plot the classification accuracy as a function of m. Add the baseline performance as a dotted horizontal line. Explain your observations on the effect of m on classification performance.

The goal of the question is to classify these numbers using the 1-NN (nearest neighbor) algorithm and then see how this performance changes using PCA (Principal Component Analysis).

The effect of changing the number of principal components m in the PCA + reconstruction + 1-NN process is being examined in the code. The result graph obtained from the code is shown below:



This graph shows that an increase in the m value increases the accuracy margin up to values around 100, and after values around 100, it decreases slightly and then stabilizes at a point around values around 450.

The reason for this is that working with too many components for PCA actually reduces accuracy after a point and causes the code to return to the accuracy rate without PCA.

Exercise 4. *k*-NN Classification Implementation

In the previous task, you used a classifier from *sklearn*. Now implement your own! Please make a copy of this notebook and complete the missing methods.

1. Compute the baseline *5*-NN classification accuracy using `sklearn.neighbors.KNeighborsClassifier`. Report the percentage of correctly classified test samples (*accuracy*).
2. Complete the method: `def predict_proba(self, X)`. Constraints: use only *numpy* (no `sklearn` inside this method), and ensure correctness for any number of neighbors.
3. Evaluate your implementation and report its accuracy for $k = 5$. *Tip: it should be close to the* `sklearn` *baseline.*
4. Compare the speed of your algorithm with `sklearn` on the same data.
5. Speed-up with KDTree: Use *KDTree* from `sklearn` to accelerate neighbor queries. Complete the missing lines, and compare accuracy & speed with brute-force implementation.
6. *(optional)* If you have time and desire, you can try to implement *KDTree* algorithm. You may find the following links useful:
   - How to Search Data with KDTree
   - A look into K-Dimensional Trees